

Connect 4

Overview:

We created a connect 4 game that makes the user play against the ai while the ai maximizes the score and the user minimizes it and all the game logic was implemented calculating the score as the game progresses.

connect4.py:

We implemented all the logic for the game in this class including how the ai chooses its move according to the game status to maximize its winning score to be able to win while also choosing which search algorithm to use.

Gui.py:

A gui was implemented to visualize the game as a grid is shown as the board for the game while the grid is updated automatically and a start button was used to start the game, a drop down menu was used to choose the algorithm in which the ai will search for the best moves.

Data structures used:

Classes: classes was used as in connect 4 class that has the board initialization and all the game logic while a node class was used to initialize each node.

Lists: list was used as in a list to store all the children for the search tree

Tuples: tuples were used as in the number of fours for each the player and the ai

Algorithms used:

Minimax: A decision-making algorithm used in games, where two players take turns. It assumes both play optimally: one (maximizer) tries to maximize the score, while the other (minimizer) tries to minimize it. The algorithm evaluates possible moves to pick the best one. Where the maximizer is the ai

Expect minimax: A variation of Minimax for games involving chance (e.g., dice). Instead of assuming an opponent's optimal move, it calculates an *expected value* of outcomes, factoring in probabilities of random events.

Alpha beta pruning: An optimization for Minimax that skips branches of the game tree that don't influence the final decision, reducing computation. Alpha represents the best already found for the maximizer, while Beta is for the minimizer.

Sample runs:

